

Estruturas de Dados — Sequência Duplamente Encadeada (Java)

Sequência

Sequencia.java

```
package Implementação.Sequencia;

public interface Sequencia{
    boolean isFirst(int i);
    boolean isLast(int i);
    Object first() throws SequenciaVazia;
    Object last() throws SequenciaVazia;
    Object before(int i) throws SequenciaVazia;
    Object after(int i) throws SequenciaVazia;
    Object replaceElement(int i, Object o) throws SequenciaVazia;
    void swapElements(int i, int j) throws SequenciaVazia;
    void insertBefore(int i, Object o);
    void insertAfter(int i, Object o);
    void insertFirst(Object o);
    void insertLast(Object o);
    Object remove(int i) throws SequenciaVazia;
    int size();
    boolean isEmpty();
    Object elemAtRank(int r) throws SequenciaVazia;
    Object replaceAtRank(int r, Object o) throws SequenciaVazia;
    void insertAtRank(int r, Object o);
    Object removeAtRank(int r) throws SequenciaVazia;
}
```

SequenciaVazia.java

```
package Implementação.Sequencia;

public class SequenciaVazia extends RuntimeException{
    public SequenciaVazia(String err){
        super(err);
    }
}
```

No.java

```
package Implementação.Sequencia;

public class No {
    private Object elemento;
    private No proximo, anterior;

    public void setAnterior(No anterior){
        this.anterior = anterior;
    }

    public void setProximo(No proximo){
        this.proximo = proximo;
    }

    public void setElement(Object elemento){
        this.elemento = elemento;
    }

    public Object getElement(){
        return elemento;
    }

    public No getAnterior(){
        return anterior;
    }

    public No getProximo(){
        return proximo;
    }
}
```

SequenciaDupla.java

```
package Implementação.Sequencia;

public class SequenciaDupla implements Sequencia {

    private int size;
    private No head, tail;

    public SequenciaDupla() {
        this.head = new No();
        this.tail = new No();
        this.head.setElement(null);
    }
}
```

```

        this.tail.setElement(null);
        this.head.setProximo(tail);
        this.tail.setAnterior(head);
        this.size = 0;
    }

    public Object elemAtRank(int r) throws SequenciaVazia {
        if (isEmpty()) {
            throw new SequenciaVazia("Sequencia está vazia!");
        }

        if (r < 0 || r >= size()) {
            throw new SequenciaVazia("Indice invalido");
        }

        No elemento = head.getProximo();

        for (int i = 0; i < r; i++) {
            elemento = elemento.getProximo();
        }

        return elemento.getElement();
    }

    public Object replaceAtRank(int r, Object o) throws SequenciaVazia {
        if (isEmpty()) {
            throw new SequenciaVazia("Sequencia está vazia!");
        }

        if (r < 0 || r >= size()) {
            throw new SequenciaVazia("Indice invalido");
        }

        No elemento = head.getProximo();

        for (int i = 0; i < r; i++) {
            elemento = elemento.getProximo();
        }

        Object salvo = elemento.getElement();
        elemento.setElement(o);

        return salvo;
    }

    public Object removeAtRank(int r) throws SequenciaVazia {

```

```

    if (isEmpty()) {
        throw new SequenciaVazia("Sequencia está vazia!");
    }

    if (r < 0 || r >= size()) {
        throw new SequenciaVazia("Indice invalido");
    }

    No elemento = head.getProximo();

    for (int i = 0; i < r; i++) {
        elemento = elemento.getProximo();
    }

    No anterior = elemento.getAnterior();
    No proximo = elemento.getProximo();

    anterior.setProximo(proximo);
    proximo.setAnterior(anterior);

    size--;
    return elemento.getElement();
}

public void insertAtRank(int r, Object o) {

    No anterior = head;

    if (r < 0 || r > size()) {
        throw new SequenciaVazia("Indice invalido");
    }

    for (int i = 0; i < r; i++) {
        anterior = anterior.getProximo();
    }

    No novo = new No();
    novo.setElement(o);

    No proximo = anterior.getProximo();

    novo.setAnterior(anterior);
    novo.setProximo(proximo);

    anterior.setProximo(novo);
    proximo.setAnterior(novo);
}

```

```

        size++;
    }

    public boolean isFirst(int i) {
        return i == 0;
    }

    public boolean isLast(int i) {
        return i == size() - 1;
    }

    public Object first() throws SequenciaVazia {
        if (isEmpty()) {
            throw new SequenciaVazia("Sequencia está vazia!");
        }

        return head.getProximo().getElement();
    }

    public Object last() throws SequenciaVazia {
        if (isEmpty()) {
            throw new SequenciaVazia("Sequencia está vazia!");
        }

        return tail.getAnterior().getElement();
    }

    public Object before(int i) throws SequenciaVazia {
        if (isEmpty()) {
            throw new SequenciaVazia("Sequencia está vazia!");
        }

        if (i < 0 || i >= size()) {
            throw new SequenciaVazia("Indice invalido");
        }

        No elemento = head.getProximo();

        for (int j = 0; j < i; j++) {
            elemento = elemento.getProximo();
        }

        return elemento.getAnterior().getElement();
    }
}

```

```

public Object after(int i) throws SequenciaVazia {
    if (isEmpty()) {
        throw new SequenciaVazia("Sequencia está vazia!");
    }

    if (i < 0 || i >= size() - 1) {
        throw new SequenciaVazia("Indice invalido");
    }

    No elemento = head.getProximo();

    for (int j = 0; j < i; j++) {
        elemento = elemento.getProximo();
    }

    return elemento.getProximo().getElement();
}

public Object replaceElement(int i, Object o) throws SequenciaVazia {
    if (isEmpty()) {
        throw new SequenciaVazia("Sequencia está vazia!");
    }

    if (i < 0 || i >= size()) {
        throw new SequenciaVazia("Indice invalido");
    }

    No elemento = head.getProximo();

    for (int j = 0; j < i; j++) {
        elemento = elemento.getProximo();
    }

    Object salvo = elemento.getElement();
    elemento.setElement(o);

    return salvo;
}

public void swapElements(int i, int j) throws SequenciaVazia {
    if (isEmpty()) {
        throw new SequenciaVazia("Sequencia está vazia!");
    }

    if (i < 0 || i >= size() || j < 0 || j >= size()) {
        throw new SequenciaVazia("Indice invalido");
    }
}

```

```

    }

    if (i == j) return;

    No a = head.getProximo();
    No b = head.getProximo();

    for (int r = 0; r < i; r++) a = a.getProximo();
    for (int r = 0; r < j; r++) b = b.getProximo();

    Object temp = a.getElement();
    a.setElement(b.getElement());
    b.setElement(temp);
}

public void insertBefore(int i, Object o) {
    if (i < 0 || i > size()) {
        throw new SequenciaVazia("Indice invalido");
    }

    No atual = head.getProximo();

    for (int j = 0; j < i; j++) {
        atual = atual.getProximo();
    }

    No anterior = atual.getAnterior();
    No novo = new No();

    novo.setElement(o);

    novo.setAnterior(anterior);
    novo.setProximo(atual);

    anterior.setProximo(novo);
    atual.setAnterior(novo);

    size++;
}

public void insertAfter(int i, Object o) {
    if (i < 0 || i >= size()) {
        throw new SequenciaVazia("Indice invalido");
    }

    No atual = head.getProximo();

```

```

    for (int j = 0; j < i; j++) {
        atual = atual.getProximo();
    }

    No proximo = atual.getProximo();
    No novo = new No();

    novo.setElement(o);

    novo.setAnterior(atual);
    novo.setProximo(proximo);

    atual.setProximo(novo);
    proximo.setAnterior(novo);

    size++;
}

public void insertFirst(Object o) {
    No atual = head.getProximo();
    No novo = new No();

    novo.setElement(o);
    novo.setAnterior(head);
    novo.setProximo(atual);

    atual.setAnterior(novo);
    head.setProximo(novo);

    size++;
}

public void insertLast(Object o) {
    No atual = tail.getAnterior();
    No novo = new No();

    novo.setElement(o);
    novo.setAnterior(atual);
    novo.setProximo(tail);

    atual.setProximo(novo);
    tail.setAnterior(novo);

    size++;
}

```



```

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }
}

```

TestesSequencia.java

```

package Implementação.Sequencia;

public class TestesSequencia {
    public static void main(String[] args) {

        SequenciaDupla s = new SequenciaDupla();

        s.insertLast("A");
        s.insertLast("B");
        s.insertLast("C");

        System.out.println(s.first());
        System.out.println(s.last());
        System.out.println(s.size());

        s.insertFirst("0");

        System.out.println(s.first());

        System.out.println(s.elemAtRank(2));

        System.out.println(s.replaceAtRank(1, "X"));

        System.out.println(s.removeAtRank(2));

        System.out.println(s.size());
    }
}

```